Factorial Calculation:

◆ 1. Factorial using for loop

```
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

print(factorial(5))  # 120
```

ব্যাখ্যা:

- `result = 1` দিয়ে শুরু
- 1 থেকে n পর্যন্ত সব number গুন করা হলো
- শেষে `return result` করে value ফিরিয়ে দেওয়া হলো

◆ 2. Factorial using while loop

```
def factorial(n):
    result = 1
    while n > 0:
        result *= n
        n -= 1
    return result

print(factorial(6))  # 720
```

◆ 3. Factorial using recursion

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5))  # 120
```

ব্যাখ্যা:

- `factorial(n)` নিজেই `factorial(n-1)` কে call করছে

- Base case: `n == 0 or n == 1` → 1 return করা

◆ 4. Factorial using math module

```
import math

print(math.factorial(5))  # 120
```

✅ সংক্ষেপে:

- **Loop method** → সহজ, step by step গুন
- **Recursion** → elegant, কিন্তু বড় number এ Stack Overflow হতে পারে
- **math.factorial()** → ready-made, দ্রুত

List Manipulation

◆ 1. List তৈরি করা

```
fruits = ["apple", "banana", "cherry"]
numbers = [1, 2, 3, 4, 5]
mixed = [1, "apple", True, 3.5]
```

◆ 2. Access Elements

```
print(fruits[0])   # apple
print(fruits[-1])  # cherry (last element)
```

◆ 3. Slicing

```
print(numbers[1:4])  # [2, 3, 4]
print(numbers[:3])   # [1, 2, 3]
print(numbers[2:])   # [3, 4, 5]
```

◆ 4. Adding Elements

```
fruits.append("orange")          # list এর শেষ এ add
```

```
fruits.insert(1, "mango")        # নির্দিষ্ট index এ add
fruits.extend(["grape", "kiwi"]) # একাধিক element add
print(fruits)
```

◆ 5. Removing Elements

```
fruits.remove("banana")  # value remove
fruits.pop()             # last element remove
fruits.pop(1)            # index 1 remove
del fruits[0]            # index 0 remove
fruits.clear()           # সব element remove
```

◆ 6. Searching Elements

```
numbers = [1, 2, 3, 4, 5]
print(3 in numbers)       # True
print(numbers.index(4))   # 3 (index)
print(numbers.count(2))   # 1
```

◆ 7. Sorting / Reversing

```
numbers = [5, 2, 9, 1, 7]
numbers.sort()            # ascending
numbers.sort(reverse=True) # descending
numbers.reverse()          # list উল্টানো
```

◆ 8. Copying List

```
numbers = [1, 2, 3]
copy1 = numbers.copy()
copy2 = list(numbers)
```

◆ 9. Loop through List

```
for fruit in fruits:
    print(fruit)

for i, fruit in enumerate(fruits):
```

```
    print(i, fruit)
```

◆ 10. List Comprehension

```
numbers = [1, 2, 3, 4, 5]
squares = [x*x for x in numbers if x%2==0]
print(squares)  # [4, 16]
```

✅ **সংক্ষেপে:**

- List mutable → add/remove/modify করা যায়
- Ordered → index দিয়ে access করা যায়
- Powerful features → slicing, comprehension, methods

Problems

1) List Sum
   Write a Python function that takes a list of numbers as input and returns the sum of all the elements in the list.
   Input_numbers=[1,2,3,4,5]
   Sum:15
2) List Reversal
   Create a function that reverse a given list and returns the reversed list
   Original_list=[10,20,30,40,50]
   Reversed List:[50,40,30,20,10]
3) List Even Numbers
   Write a function that takes a list of integers as input and returns a new list containing only the even numbers
   numbers=[1,2,3,4,5,6,7,8,9,10]
   Even Numbers:[2,4,6,8,10]
4) List Duplicates
   Create a function that finds and returns a list of unique elements from a given list.
   elements=[1,2,2,3,4,4,5]
   Unique Elements:[1,2,3,4,5]

Approach 1: Using set() (simplest way)

```
def unique_elements(lst):
    return list(set(lst))  # set makes elements unique
```

```
elements = [1, 2, 2, 3, 4, 4, 5]
print("Unique Elements:", unique_elements(elements))
```

Output (unordered because set):

```
Unique Elements: [1, 2, 3, 4, 5]
```

Note: set ব্যবহার করলে order maintain হয় না।

Approach 2: Maintain original order

```
def unique_elements(lst):
    unique = []
    for item in lst:
        if item not in unique:
            unique.append(item)
    return unique

elements = [1, 2, 2, 3, 4, 4, 5]
print("Unique Elements:", unique_elements(elements))
```

Output (order preserved):

```
Unique Elements: [1, 2, 3, 4, 5]
```

Approach 3: Using dict.fromkeys() (Python 3.7+)

```
elements = [1, 2, 2, 3, 4, 4, 5]
unique = list(dict.fromkeys(elements))
print("Unique Elements:", unique)
```

Output:

```
Unique Elements: [1, 2, 3, 4, 5]
```

✅ Summary:

- `set()` → fastest, order lose হতে পারে
- Loop / `dict.fromkeys()` → order preserve

5) List Concatenation
   Write a function that takes two lists as a input and returns a new list that is the concatenation of the two input lists
   List1=[1,2,3]

List2=[4,5,6]
Concatenated List:[1,2,3,4,5,6]
List Concatenation:

◆ 1. Using `+` operator

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]

result = list1 + list2
print(result)  # [1, 2, 3, 4, 5, 6]
```

◆ 2. Using `extend()` method

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]

list1.extend(list2)  # list1 এর মধ্যে list2 যোগ হয়ে যাবে
print(list1)          # [1, 2, 3, 4, 5, 6]
```

Note: `extend()` modifies the original list, `+` operator creates a new list

---

◆ 3. Using `*` operator (Python 3.5+)

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]

result = [*list1, *list2]
print(result)  # [1, 2, 3, 4, 5, 6]
```

◆ 4. Using `itertools.chain()` (for large lists)

```
from itertools import chain

list1 = [1, 2, 3]
list2 = [4, 5, 6]

result = list(chain(list1, list2))
print(result)  # [1, 2, 3, 4, 5, 6]
```

✅ **Summary / Key Points**

- `+` → নতুন list তৈরি করে
- `extend()` → original list modify করে
- `*` → Python 3.5+ flexible
- `itertools.chain()` → efficient for large lists

Parameters vs Arguments:

◆ 1. Parameter কী?

- **Parameter** হলো **function define করার সময়** function-এর ভিতরে লেখা variable।
- এটি **placeholder** হিসেবে কাজ করে, যেটার মধ্যে value pass হবে।

```
def greet(name, age):   # name & age হলো parameters
    print(f"Hello {name}, you are {age} years old.")
```

◆ 2. Argument কী?

- **Argument** হলো **function call করার সময়** যেই actual value আমরা function-এ pass করি।

```
greet("Nusrat", 28)   # "Nusrat" এবং 28 হলো arguments
```

◆ 3. তুলনা / সংক্ষেপে

Feature	Parameter	Argument
কোথায় define হয়	Function definition-এ	Function call-এ
কাজ	Placeholder / variable	Actual value যা pass করা হয়
উদাহরণ	`name, age`	`"Nusrat", 28`

◆ 4. উদাহরণ: সব ধরণের argument

```
# function definition with parameters
def student_info(name, age=18, *subjects, **extra):
    print("Name:", name)
```

```
    print("Age:", age)
    print("Subjects:", subjects)
    print("Extra Info:", extra)

# function call with arguments
student_info("Nusrat", 20, "Math", "Physics", grade="A",
country="Bangladesh")
```

Output:

```
Name: Nusrat
Age: 20
Subjects: ('Math', 'Physics')
Extra Info: {'grade': 'A', 'country': 'Bangladesh'}
```

- এখানে:
  - `name`, `age`, `*subjects`, `**extra` → **Parameters**
  - `"Nusrat"`, `20`, `"Math"`, `"Physics"`, `grade="A"`, `country="Bangladesh"` → **Arguments**

Keyword Argument

def personal(job, income):

print("Job: ",job)

print("Income: ",income)

personal(income=2000000, job="Data Science")

Output:

Job: Data Sciennce

Income: 2000000

কী ওয়ার্ড দিয়ে দেয়ার কারণে প্যারামিটার আর আর্গুমেন্টেরব পজিশন ভিন্ন হওয়া সত্বেও সঠিক সিকুয়েন্সে আউটপুট দিয়েছে।

Variable length argument:

def add_num(*x):

print(x)

return sum(x)

y=add_num(10,20,30,40,50)

print(y)

Output:

(10,20,30,40,50)

150

recursion:

◆ 1. Recursion কী?

Recursion হলো এমন একটি প্রোগ্রামিং technique যেখানে **একটি function নিজেকেই call করে**।

- এটি ব্যবহার হয় **problems যা ছোট sub-problems এ ভাগ করা যায়**।
- খুব common examples: factorial, Fibonacci, sum of list, tree traversal ইত্যাদি।

◆ 2. Recursion এর Structure

একটি recursive function সাধারণত দুইটি অংশ থাকে:

1. **Base Case** → কখন function আর নিজেকেই call করবে না (stop condition)
2. **Recursive Case** → function নিজের call করে ছোট version of problem এ

```
def function_name(parameters):
    if base_condition:
        return something
    else:
        return function_name(smaller_problem)
```

◆ 3. Example 1: Factorial using Recursion

```
def factorial(n):
    if n == 0 or n == 1:   # base case
        return 1
    else:
```

```
        return n * factorial(n - 1)   # recursive call

print(factorial(5))  # 120
```

Flow:

```
factorial(5) = 5 * factorial(4)
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1   # base case
```

◆ 4. Example 2: Sum of numbers using Recursion

```
def sum_numbers(n):
    if n == 0:   # base case
        return 0
    else:
        return n + sum_numbers(n - 1)

print(sum_numbers(5))  # 15 (5+4+3+2+1)
```

◆ 5. Example 3: Fibonacci Series using Recursion

```
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n-1) + fibonacci(n-2)

for i in range(6):
    print(fibonacci(i), end=" ")  # 0 1 1 2 3 5
```

◆ 6. Key Points about Recursion

1. Always have **base case** → না হলে infinite recursion হবে (RecursionError)
2. Each recursive call **stack memory** use করে → বড় input হলে memory overflow হতে পারে

3. Recursion elegant, কিন্তু loop (iteration) অনেক সময় faster হয়

lambda in Higher Order Function:

◆ 1. Higher-Order Function কী?

- **Higher-order function** হলো এমন function যা:
  1. অন্য function কে **argument হিসেবে নেয়**
  2. অন্য function **return করে**
- উদাহরণ: `map()`, `filter()`, `reduce()`, custom function

◆ 2. Lambda function কী?

- **Lambda function** = এক লাইনের anonymous function
- Syntax:

```
lambda arguments: expression
```

◆ 3. Example 1: Using lambda inside map (Higher-Order Function)

```
numbers = [1, 2, 3, 4, 5]

# map() হল higher-order function, lambda হল anonymous function
squares = list(map(lambda x: x*x, numbers))

print(squares)  # [1, 4, 9, 16, 25]
```

ব্যাখ্যা:

- `map()` → higher-order function
- `lambda x: x*x` → function pass করা হলো
- সব element-এ apply করে new list return করলো

◆ 4. Example 2: Custom Higher-Order Function

```
def apply_func(func, value):
    return func(value)   # func হল argument function

# lambda function pass করা হলো
result = apply_func(lambda x: x + 10, 5)
print(result)  # 15
```

ব্যাখ্যা:

- `apply_func()` → higher-order
- `lambda x: x+10` → function হিসেবে argument
- 5 pass হয়ে 15 return হয়েছে

◆ 5. Example 3: Returning Lambda (function returning function)

```
def multiplier(n):
    return lambda x: x * n  # lambda function return হচ্ছে

double = multiplier(2)
triple = multiplier(3)

print(double(5))  # 10
print(triple(5))  # 15
```

ব্যাখ্যা:

- `multiplier(2)` → returns `lambda x: x*2`
- `double(5)` → 5*2 = 10

◆ 6. Summary / Key Points

Concept	Example

Higher-Order Function	Function takes/returns another function
Lambda	One-line anonymous function
Lambda + HOF	Pass lambda as argument OR return lambda from HOF
Examples	`map(lambda x: x*2, list)`, `apply_func(lambda x:x+10,5)`, `multiplier(2)`

module:

◆ 1. Module কী?

- **Module** হলো **Python file (.py)** যা **কোডের collection** রাখে।
- Module এ functions, classes, variables থাকতে পারে।
- Modules ব্যবহার করে আমরা **code reuse** এবং **structure** করতে পারি।

◆ 2. Python built-in modules

Python অনেক built-in modules দিয়ে আসে, যেমন:

Module	ব্যবহার
math	Mathematical functions
random	Random numbers
datetime	Date & time handling
os	Operating system
sys	System-specific info

◆ 3. Module import করার উপায়

1) Import whole module

```
import math
```

```
print(math.sqrt(16))  # 4.0
```

2)Import specific function / class

```
from math import sqrt, factorial

print(sqrt(25))       # 5.0
print(factorial(5))   # 120
```

3) Import with alias

```
import math as m

print(m.ceil(4.2))   # 5
```

◆ 4. Custom Module তৈরি করা

1. Create a file `mymodule.py`:

```
# mymodule.py
def greet(name):
    return f"Hello, {name}!"

pi = 3.14159
```

2. Import এবং ব্যবহার:

```
import mymodule

print(mymodule.greet("Nusrat"))  # Hello, Nusrat!
print(mymodule.pi)               # 3.14159
```

◆ 5. Module ব্যবহার করার সুবিধা

- Code reuse করা যায়
- Project organized থাকে
- Built-in modules ব্যবহার করলে time save হয়